

Line Follower Robot with PID Adaptive Cruise Control

Aliye BİRGÜL
Department of Electrical
and Electronics Engineering
Abdullah Gül University
Kayseri/TURKEY
aliye.birgul@agu.edu.tr

Samet Can ŞAHİN
Department of Electrical
and Electronics Engineering
Abdullah Gül University
Kayseri/TURKEY
sametcan.sahin@agu.edu.tr

Serkan KIYAKLI
Department of Electrical
and Electronics Engineering
Abdullah Gül University
Kayseri/TURKEY
serkan.kiyakli@agu.edu.tr

Zeki BOZDOĞANLI
Department of Electrical
and Electronics Engineering
Abdullah Gül University
Kayseri/TURKEY
zeki.bozdoganli@agu.edu.tr

Abstract—This project was carried out with the aim of designing a line-following robot operating with PID control. Line-following is a common and reliable method that allows mobile robots to navigate in a controlled environment via a distinct line or track. In the project, an infrared (IR) sensor was used for the robot to scan a surface, and an ultrasonic distance sensor was used to follow an object by maintaining a certain distance. The robot was enabled to follow the line and follow a plate by maintaining a certain distance. In this context, the design and implementation of the robot, setting the controller parameters, drawing circuit diagrams, writing the microcontroller-based embedded code and optimizing the system performance were aimed. In order to ensure the success of the project, the robot's line-following performance and distance-keeping ability with the plate were tested and the final design was created.

Keywords— *Controlled Environment, Controller parameters, Distance maintenance, Embedded code, Infrared (IR) sensor, Line-following, Line-following robot, Microcontroller-based system, Mobile robots, PID control, Plate tracking, System optimization, Ultrasonic distance sensor.*

I. INTRODUCTION

Autonomous robotic systems are used in many areas, from industrial automation to personal assistants. Among these systems, line-following robots stand out with their ability to navigate structured environments. Line-following technology allows robots to detect and follow predetermined paths using sensor data. This technology is especially preferred in areas such as logistics, production and warehouse automation due to its reliability and adaptability.

One of the most important features of line-following robots is their ability to maintain stability and accuracy while adapting to changing scenarios. In this context, PID (Proportional-Integral-Derivative) control systems are widely used in robotic applications thanks to their ability to precisely control motor speeds and motion corrections. PID control allows the robot to follow the line stably and respond quickly to environmental changes.

This project aims to design and implement a line-following robot with PID control. The robot uses an infrared (IR) sensor to detect a surface and an ultrasonic sensor to follow a moving object while maintaining a certain distance. These sensors are combined in an integrated embedded control system. The STM32F103RB with C register base was used for processing the data obtained from the sensors. The project aims to address the challenges of real-time line tracking and adaptive distance control by combining

hardware design, implementation of control algorithms and system optimization.

The methods adopted in this project are compatible with these principles and aim to ensure that the robot operates effectively under different conditions. Iterative tests and parameter adjustments were performed to improve the performance of the system, and the findings obtained from this process shed light on the practical challenges in the field of autonomous robotics.

This study presents the design process, implementation stages and test results of the project. It also contributes to the development of reliable and efficient robotic solutions by discussing the practical applications and limitations of the proposed system.

II. DESIGN AND PROCEDURES

A. COMPONENTS

i. Microcontroller (STM32F103RB): Microcontroller (STM32F103RB): In this project, the STM32F103RB microcontroller was chosen for the purpose of implementing PID control algorithms and optimizing the performance of the robot. Built on the ARM Cortex-M3 core and operating at 72 MHz, this microcontroller has the capacity to perform fast and precise operations, which provides a critical advantage for real-time control applications. While 128 KB Flash memory and 20 KB SRAM provide sufficient space for storing embedded system code and data processing, various peripherals such as USB, CAN, ADC and timers provide flexibility in tasks such as sensor integration and motor control. In particular, dual ADC units enable the robot to process infrared and ultrasonic sensor data simultaneously, allowing it to respond faster and more accurately. In addition, thanks to low power consumption, energy efficiency is provided, extending the robot's operating time, and a reliable solution is offered with its industrial-level durability. All these features make the STM32F103RB an ideal microcontroller for this project.

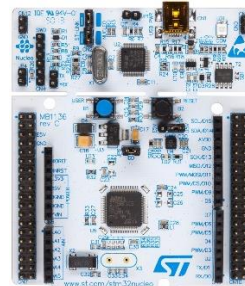


Fig 2.1: The microcontroller of STM32F103RB.

ii. DC Motor: In this project, DC motors are used to provide the movement of the robot. In this project, DC motors were used to provide the movement of the line-following robot with a PID controller. The speeds of the DC motors were adjusted with PID according to the road and obstacles by connecting them to the wheels.



Fig 2.2: The DC motor.

iii. DC Motor Driver (L298N): The L298N motor driver is used to convert low power signals from the microcontroller to the higher current and voltage levels required for DC motor operation. The L298N is an H-bridge motor driver that can control the speed and direction of two independent DC motors. This feature allows the robot to move accurately on the track and maintain the desired distance.

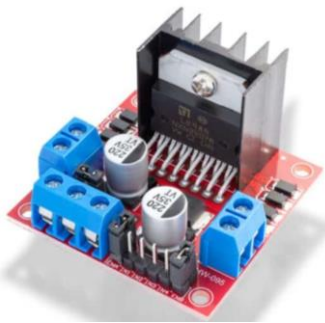


Fig 2.3: The L298N motor driver.

iv. Infrared (IR) Sensor (QTR8A): The QTR-8A infrared sensor used in this project is used for the robot to detect and follow the line. It detects the difference between light and dark colors on the surface thanks to its eight independent sensors. Its analog outputs are integrated with the PID control algorithm, thus dynamically adjusting the motor speeds and ensuring stability on the line.



Fig 2.4: The IR Sensor (QTR8A).

v. Ultrasonic Sensor (HC-SR04): The HC-SR04 ultrasonic sensor was used to measure the distance to objects in front of the robot and maintain a certain distance. The sensor calculates the distance by sending ultrasonic waves and measuring the return time. This feature helps the robot decide whether to move or not according to the object in front of it.



Fig 2.5: The ultrasonic sensor (HC-SR04)

vi. LCD screen (QAPAS1602): The LCD screen visually presents the operating status of the robot, sensor data and system parameters to the user. The screen provides real-time monitoring of information such as distance, plate setting parameters.



Fig 2.6: The LCD screen.

vii. Buzzer: The buzzer was used to give audible alerts in case of errors or when certain conditions are met in this project. It was used to inform the user quickly and effectively.



Fig 2.7: The buzzer.

viii. LED: In this project, LED was used to illuminate the surroundings.



Fig 2.8: LED.

B. DESIGN OF THE SYSTEM

i. Hardware Part:

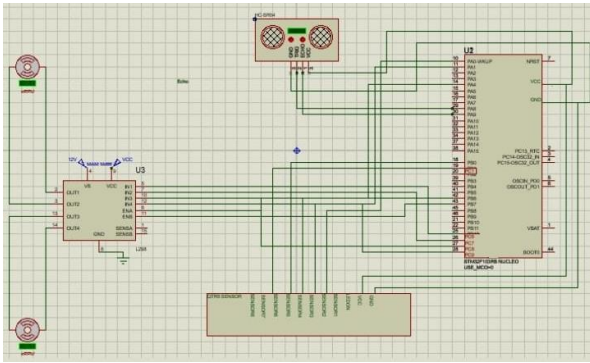


Fig 2.9: The circuit schematic.

The connections of the STM32F103RB microcontroller are set up to make all system components work together properly. For the distance sensor, the Trig pin was connected to the PA8 pin, and the Echo pin was connected to the PA9 pin. The Trig pin sends a trigger signal to the sensor and emits a sound wave. The Echo pin detects the echo signal returned from the sensor and allows the distance to be calculated. This configuration allows the microcontroller to detect obstacles with the help of an ultrasonic sensor.

In the connections of the motor drivers, the EnA (speed control) pin of Motor A was connected to PB6. This pin controls the speed of the motor by sending PWM (Pulse Width Modulation) signals. In1 and In2 pins (connected to PC6 and PC7 respectively) allow the motor to move in the forward or reverse direction. Similarly, for Motor B, the EnB pin is connected to PB7, and the direction control pins In3 and In4 were connected to PC8 and PC9 respectively. This configuration allows independent speed and direction control of both motors. Connecting the motor drivers in this way increases the mobility and precision of the robotic systems.

The QTR-8 IR sensor was connected to various pins of the STM32F103RB. This sensor contains optical sensors that detect light intensity for the robot to follow the line. The second sensor was connected to pins PA0, the third sensor to PA1, the fourth sensor to PA4, the fifth sensor to PB0, and the sixth sensor to PC1. These connections allow the microcontroller to read the output of each sensor and determine the position of the line. Connecting these sensors correctly allows the robot to follow the line quickly and stably.

A 5V power supply is provided for all components and all elements are connected to a common ground (GND). This is critical for the system to operate stably and to equalize the voltage levels. As a result, these connections configure the STM32F103RB microcontroller to seamlessly perform functions such as distance sensing, motor control, and line tracking.

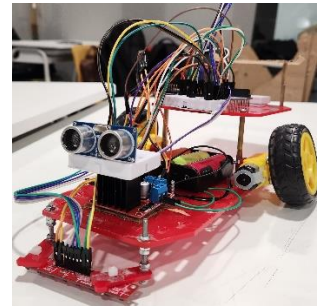


Fig 2.10: The general structure of line follower.

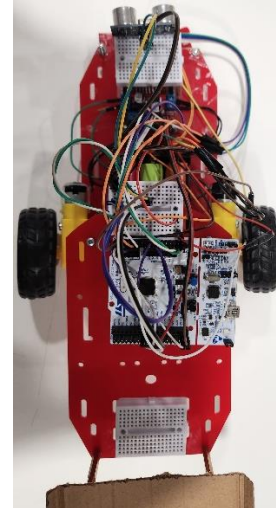


Fig 2.11: The microcontroller of STM32F103RB.

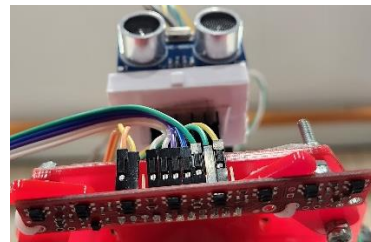


Fig 2.12: The IR and distance sensors.

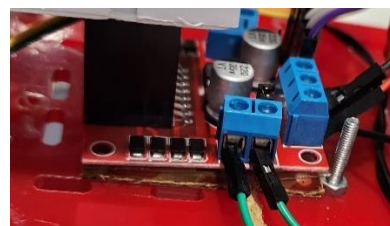


Fig 2.13: The L298N motor driver.

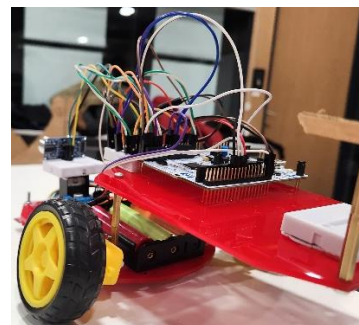


Fig 2.14: Background of the line follower robot.

ii. Theoretical Calculations:

A. Calculation of Positional Error:

The error is calculated by analyzing the data collected from the sensors. Each sensor reading provides positional data, determined using formula (1). Using this positional data, the error is computed with formula (2).

$$x = \frac{\sum_{i=0}^{N-1} i * 1.03^{(sensordata[i]-285)*1.2}}{\sum_{i=0}^{N-1} 1.03^{(sensordata[i]-285)}} \quad (1)$$

$$e(t) = \left(\frac{N-1}{2} - \frac{\sum_{i=0}^{N-1} i * 1.03^{(sensordata[i]-285)*1.2}}{\sum_{i=0}^{N-1} 1.03^{(sensordata[i]-285)}} \right) * 20 \quad (2)$$

B. Proportional Control Constant:

Equation (3) represents the mathematical model for proportional control. It has been observed that using only proportional control allows the robot to respond more quickly to errors during line tracking. However, as the k_p gain increases beyond a certain threshold, significant oscillations occur. Conversely, reducing the k_p gain decreases oscillations but slows down the robot's response to errors. To address this, derivative control was incorporated into the system.

$$u_p(t) = k_p * e(t) \quad (3)$$

C. Integral Control Constant:

Using proportional control alone has been observed to result in a steady-state error. The mathematical expression for integral control was provided in Equation (4). The integral term accounts for the accumulation of past errors. By incorporating the k_I integral gain, the integral feedback reduces the steady-state tracking error by introducing a pole at the origin.

$$u_I(t) = k_I \int_{t_0}^t e(t) dt \quad (4)$$

During the system identification phase, it was observed that the unity feedback implementation without a PID controller inherently possessed at least one pole at the origin. However, the system was unable to accurately follow curved paths without exhibiting a steady-state error. To address this, an integral term was introduced to the controller to minimize the steady-state tracking error on curved paths. Nonetheless, summing all past errors from the start proved ineffective, as the path's shape is not always linear. Consequently, the classical integral control was modified as illustrated in Equation (5).

$$u_I(n) = k_I \sum_{i=0}^{N-1} e(n) \quad (5)$$

D. Derivative Control Constant:

Equation (6) represents the mathematical model for classical derivative control. Derivative control is introduced to reduce oscillations and overshooting caused by the proportional and integral components.

$$u_D(t) = k_D \frac{de(t)}{dt} \quad (6)$$

Derivative (k_D) control helps prevent overshooting in the system and enhances its transient response. However, it is important to note that k_D control is highly sensitive to noise. This sensitivity arises because k_D control functions as a high-pass filter, amplifying rapid changes in the error. During the robot's development, the impact of noise was minimal. To address this issue, the classical derivative control was modified, as shown in Equation (7), by calculating the difference between the current and previous errors.

$$u_D(n) = k_D (e(n) - e(n-1)) \quad (7)$$

E. Modified PID Controller:

Equation (8) shows classical PID control. Incorporating the previously discussed modifications, the complete mathematical expression for the PID controller was presented in Equation (9). In this controller, the proportional term reduces positional error, the integral term minimizes steady-state error on curved paths, and the derivative term mitigates oscillations and overshooting.

$$\begin{aligned} u(n) &= u_D(t) + u_P(t) + u_I(t) \\ &= k_P * e(t) + k_I \int_{t_0}^t e(t) dt + k_D \frac{de(t)}{dt} \end{aligned} \quad (8)$$

$$= k_P * e(n) + k_I \sum_{i=0}^{N-1} e(n) + k_D (e(n) - e(n-1)) \quad (9)$$

F. Closed Loop Controller:

The line-following robot's closed-loop control system is primarily based on a PID (Proportional-Integral-Derivative) controller, ensuring accurate and adaptive path tracking. The process begins with the QTR-8 sensor array, which detects the line by measuring reflected light intensity and converting the analog signals into digital values. The robot calculates the error term $e = 40 - A$, where A is the weighted average of the sensor readings, representing the robot's deviation from the line's center. This error is processed by the PID controller, which computes the proportional, integral, and derivative components to dynamically adjust the motor speeds.

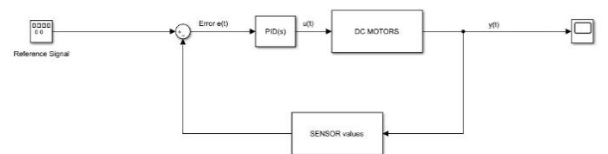


Fig 2.15: The block diagram of the system.

The proportional term addresses the current error, the integral term corrects accumulated errors over time, and the derivative term prevents overshooting by accounting for the error's rate of change. These adjustments are implemented via Pulse Width Modulation (PWM), enabling precise motor control and maintaining alignment with the line.

Additionally, the HC-SR04 ultrasonic sensor enhances safety by measuring distances and avoiding collisions. If an obstacle is detected within 20 cm, the robot stops by setting both motor speeds to zero. While the PID controller is effective on straight paths and gradual curves, it faces challenges with sharp turns where the sensors may lose the line.

In such situations, an open-loop control mechanism is activated. When no sensors detect the line, the robot relies on the outermost sensors (sensor 1 and sensor 8) and the last recorded error to execute a turn. For sharp turns (e.g., 45° or 90°), the inner wheel moves backward at 53% speed while the outer wheel moves forward at 70%, facilitating quick realignment. For gentle curves, the inner wheel rotates backward at 20% speed while the outer wheel runs at full speed for smoother navigation.

By combining the precise line-tracking capabilities of the PID controller with the open-loop mechanism for handling sharp turns, the robot achieves robust performance across different path geometries. The integration of ultrasonic sensing further enhances its adaptability and efficiency by incorporating obstacle avoidance.

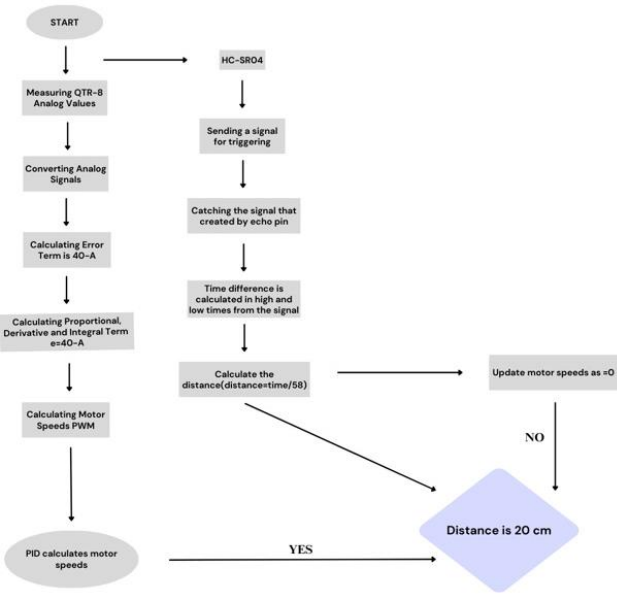


Fig 2.16: Flowchart of the system.

III. RESULT

The results of the Line Follower Robot with PID project highlight the significant influence of PID parameters on the robot's performance in both line tracking and maintaining the desired distance from a moving board. During testing, it was observed that increasing k_p improved the robot's responsiveness to deviations, allowing for quicker corrections. However, excessive k_p values led to oscillatory behavior, causing the robot to deviate and destabilize. Conversely, lower k_p values provided smoother movements but reduced sensitivity to errors, resulting in delayed corrections and difficulties in navigating sharp curves.

Similarly, k_i played a critical role in addressing steady-state errors by ensuring the robot maintained a consistent trajectory. While higher k_i values effectively corrected long-term deviations, they also caused overshooting and slower stabilization due to accumulated errors. Lower k_i , on the other hand, limited the robot's ability to handle drifts, particularly on curved tracks. The derivative component, k_d , enhanced the system's response to sudden changes, allowing the robot to handle sharp turns more effectively. However, higher k_d values introduced instability and frequent

overshooting, whereas lower k_d values resulted in smoother operation but reduced responsiveness to rapid directional changes.

The PID parameters were iteratively tuned to achieve optimal values of $k_p = 1.5$, $k_i = 0.00014$, and $k_d = 60$. These adjustments ensured stability and responsiveness in both simulation and physical tests, though discrepancies arose due to sensor noise and mechanical mismatches. Noise filtering techniques, such as moving average filtering, were applied to mitigate these issues, and calibration of the IR sensor array further improved detection accuracy under varying lighting conditions.

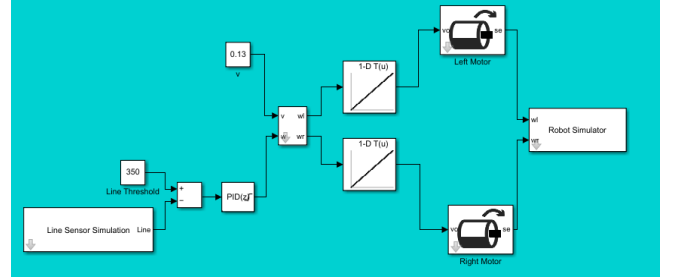


Fig 3.1: The full block diagram of the PID system.

The robot was tested on tracks featuring straight lines, curves, and sharp turns, as well as in scenarios requiring distance maintenance using an ultrasonic sensor. It consistently maintained a specified distance of 20 cm from the moving board, dynamically adjusting motor speeds in real-time. Despite sensor sensitivity variations, calibration ensured reliable performance. Physical tests validated theoretical findings, demonstrating smooth transitions, minimal overshooting, and effective handling of sharp turns, even with fluctuating board speeds.

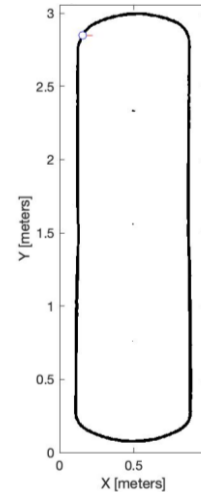


Fig 3.2: Test line for the line follower robot.

A block diagram illustrating the system components provided insight into the robot's operation. The Line Sensor Simulation generated line detection data, which was processed using a Line Threshold to calculate deviations. The PID(z) block applied the PID control algorithm to compute corrections, which dynamically adjusted the speeds of the left and right motors. These adjustments ensured accurate alignment with the line while maintaining a base speed (V).

The system characteristics were first identified using a unity feedback implementation to establish a baseline for stability before deploying the PID controller.

In conclusion, the project successfully demonstrated the implementation of a PID-controlled line-following robot capable of accurate path tracking and consistent distance maintenance, validating the effectiveness of iterative PID tuning and calibration techniques.

IV. CONCLUSION AND DISCUSSION

In this project, a line-following robot was designed in embedded C language using QTR8 IR sensor, L298N motor driver and two DC motors. No ready-made library was used in the project, all codes were written at the most basic level, i.e. directly at the register level. Although the robot successfully followed the line as designed, various technical difficulties were encountered during the project. The first difficulty was to correctly process the data received from the IR sensors. Since IR sensors produce values based on light reflection, optimizing the received data and eliminating debounce (vibration) problems was a critical process. In addition, developing an algorithm that calculates the average value with sensor data was another time-consuming challenge.

After optimizing the sensor data, the development of the PID (Proportional-Derivative-Integral) algorithm was one of the most complex stages of the project. This algorithm processed the constantly changing error values and ensured that the motors rotated in the correct direction and speed. Optimization problems encountered in sharp turns were another challenge of the project. The robot frequently went off the road and exhibited unexpected movements during this process. However, these problems have been solved, and the robot has been optimized to operate with stable performance as desired.

Significant gains have been made from this project. Technical skills such as register-based coding at the microcontroller level, processing of IR sensor data, DC motor control and development of PID algorithm have been acquired. In addition, comprehensive information has been provided on topics such as timers, PWM and use of peripherals in embedded C language. In order to take the project to further levels, it is recommended to use an IR sensor that can make more precise measurements, to prefer more efficient DC motors and to increase the functionality of the robot with additional features. As a result, the robot has successfully followed the line, provided all system optimizations and completed the targeted path in a short time.

V. LINKS

GitHub Link: <https://github.com/serkan10000001/projects>

Video Link:

<https://www.youtube.com/watch?v=hcItMWRxi00>

REFERENCES

1. Direnc.net. (n.d.). HC-SR04 ultrasonik sensör. Retrieved from <https://www.robotzade.com/urun/hc-sr04-ultrasonik-sensor>
2. Teesmaxx. (n.d.). DC redüktörlü motor 6V 250 RPM. Retrieved from <https://www.teesmaxx.com/dc-reduktorlu-motor-6v-250-rpm-dc-m>
3. Direnc.net. (n.d.). L298 DC/step motor sürücü. Retrieved from <https://www.direnc.net/l298-dc-step-motor-surucu-1>
4. STMicroelectronics. (n.d.). NUCLEO-F103RB. Retrieved from <https://www.st.com/en/evaluation-tools/nucleo-f103rb.html>
5. Direnc.net. (n.d.). QTR-8A analog kızılötesi sensör devresi. Retrieved from <https://www.direnc.net/qtr-8a-analog-kizilotesi-sensor-devresi-8li-analog-cizgi-sensoru-pololu>
6. Direnc.net. (n.d.). 2x16 LCD ekran (sol üst mavi). Retrieved from <https://www.direnc.net/2x16-lcd-display-sol-ust-mavi-qapass>
7. Direnc.net. (n.d.). 3V 12mm buzzer. Retrieved from <https://www.direnc.net/3v-12mm-2300300hz-h95mm-devreli-buzzer>